

Team Stitch



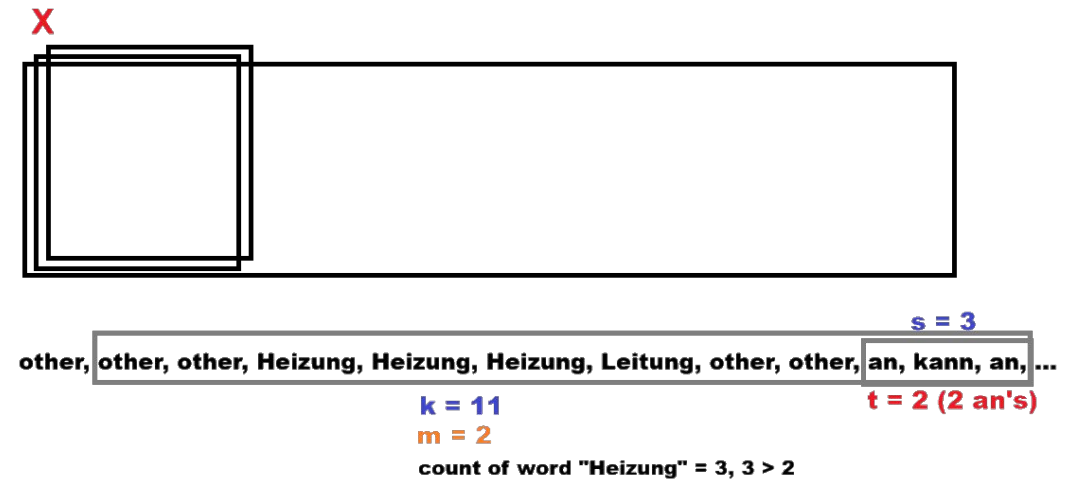
Roman Chervinskyy, Daniel Serdeshnyi, Andrei Vasiliuk

Sliding window

We implemented sliding window the following way:

Model goes through each time step and predicts a word, (we can set step to be more than 1), we name it **x**, it has a running window of size **s** in which command word “aus” or “an” must occur **t** times. If it occurred, then we look at last **k** steps and look for the most frequent word from “words with meaning”, if there is one and it occurred more than **m** times, then we say that phrase word + activation word was said.

x, **s**, **t**, **k**, **m** are all adjustable hyperparameters.



architecture illustration

Sliding window performance and hyperparameters

For model we used a combination of **ResNeXt** on melspects and simple **CNN** on additional data. After we adjusted hyperparameters for our model in task 3 we worked on hyperparameters on Sliding Window.

The best tradeoff between complexity and speed of the algorithm was step size **x = 2**. After data investigation we took **k = 28**: We took upper outlier of distance between 2 words. For **t,s,m** variables the best tuning is on the table. The best score is **-221.7**. with **m = 2, t = 2, s = 3**.

m	metrics	(t,s)			
		(1,1)	(2,3)	(3,5)	(3,4)
1	Recall	0.5946	0.5702	0.5414	0.5399
	Precision	0.8221	0.8539	0.8798	0.8818
	Cost f.	-179.7	-203.1	-207.8	-209.3
2	Recall	0.5403	0.5218	0.4999	0.4983
	Precision	0.8868	0.9105	0.9246	0.9294
	Cost f.	-215.9	-221.7	-204.3	-202.4
3	Recall	0.4579	0.4471	0.4311	0.4294
	Precision	0.9252	0.9367	0.9501	0.9533
	Cost f.	-138.2	-133	-112.2	-110.3

m - min occurrences of a device work out of last k frames. **(t, s)** is amount of an activation word **(t)** needed to be said in a sliding window of size **(s)**.

YOLO

Further we decided to try out the detection approach. For that we used YOLOv5.

Problem: We can't do it straightforwardly, because our spectrogram includes not solely the word, but also some “empty” frames before and after the word.

To handle it we calculated the mean of each timestamp and after selecting gradient peaks cropped our word.

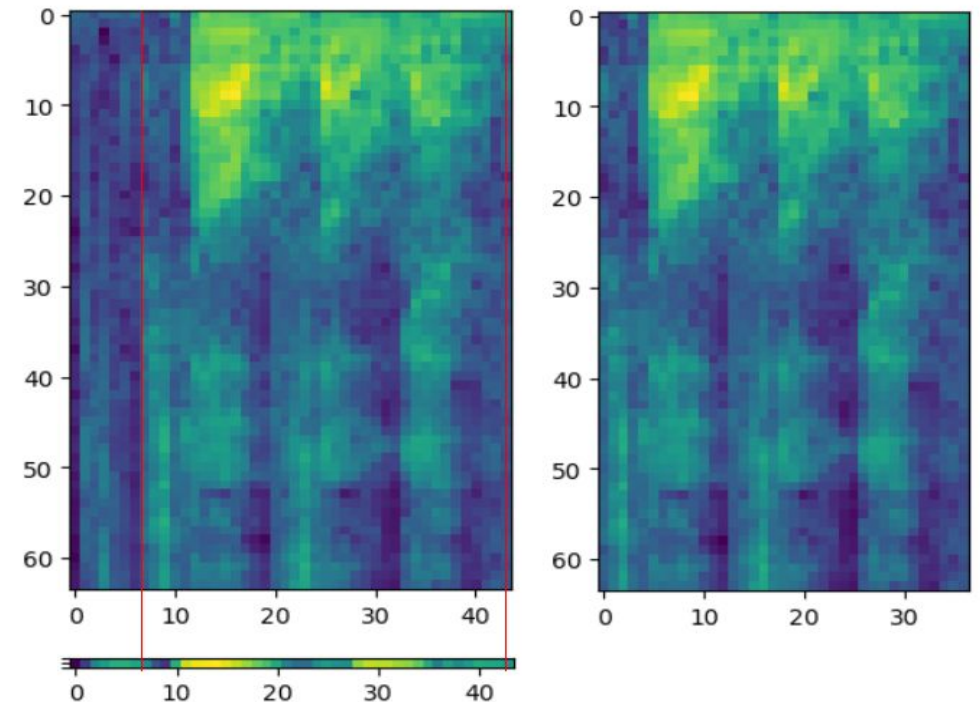


Figure [1] process of cropping a word.

YOLO Data Preprocessing

Furthermore, since the data given is not similar to the one we are given for testing we decided to artificially create training data.

We did it speaker-wise. We took random noise of arbitrary length from 1 to 44 and placed it in between the words said by this user and created phrases. With those samples we tried to imitate real-life scenarios. Our data after Data Preprocessing can be seen on the Figure [2].

Final result of YOLO with this type of data preprocessing is: **-15**.

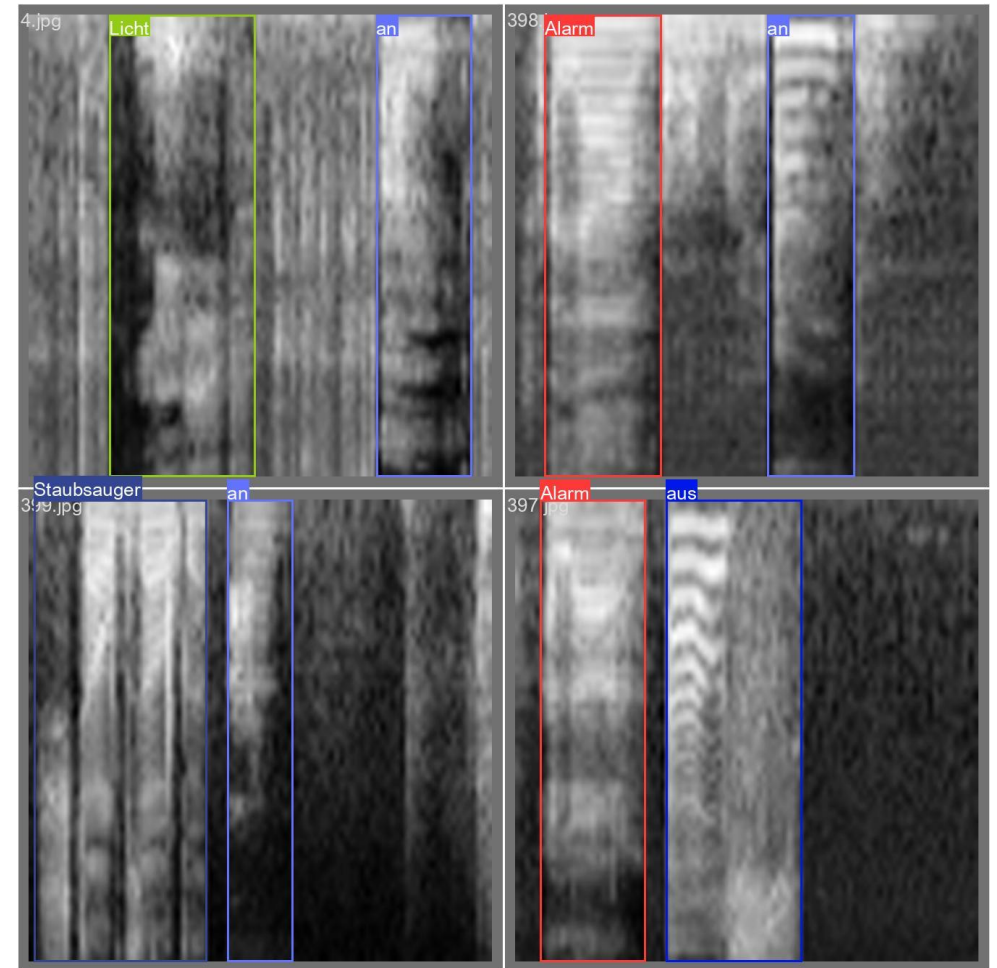


Figure [2] - data after Data Preprocessing

Overall results

1. YOLO performed poorer than common Sliding Window, but we assume that it is cause of data specifics. We believe that with more real-life-like data we could further improve our model and it would work without big flaws.
2. Our model might be too complex for a smart-home system, therefore we also would need to make it more simple.
3. YOLO approach is more stable and scalable than the sliding window and can be further improved by introducing autoregression learning and data augmentations.



Figure [3] : YOLO on images